SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING PRACTICAL ASSIGNMENT

1. Student Details

Student Name	Pratik Akrur Pandit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	26/10/2026

2. Practical Details

Practical No.	10
Practical Title	Templates (Function Template and Class Template)
Date Performed	24/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

Write a C++ program that implements function templates and class templates. The program must create generic functions (maximum, swap and a two-type function) and generic classes (Calculator and Stack) that work with different data types such as int, double, char and string without rewriting the code for each type.

4. Objectives

1. To understand templates as the foundation of generic programming in C++.
2. To implement function templates that work with different data types.
3. To implement class templates and create objects of the class for different data types.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Write a function template using the template keyword and call it with different data types.
2. Define a class template and instantiate it using the syntax ClassName<type>.
3. Explain how templates reduce code duplication and promote code reuse.

6. Theory / Concept

A **template** is a feature of C++ that allows a function or a class to work with any data type. Instead of writing the same code again and again for int, float, double, string and so on, the programmer writes it once using a generic type parameter, and the compiler generates the required version when the template is used. This is called **generic programming**. Templates are resolved at compile time, so they are a form of compile-time polymorphism.

A. Function Template

- A function template defines a family of functions that perform the same operation on different data types.

- Syntax: **template <typename T> return_type function_name(T parameter)**. The keywords typename and class can be used interchangeably for the type parameter.
- The compiler deduces the type T from the arguments of the call (for example, maximum(10, 20) uses T = int).
- A function template can take more than one type parameter, such as template <typename T, typename U>.

B. Class Template

- A class template defines a generic class whose data members and member functions can use a generic type.
- Syntax: **template <class T> class ClassName { ... };**
- Unlike function templates, the type must be given explicitly when an object is created, for example Calculator<int> obj(20, 4);
- Each different type used creates a separate class (an instantiation) generated by the compiler.

Advantages of templates: code reusability, type safety (checked at compile time), less code duplication and easier maintenance.

7. Algorithm / Pseudocode

```
START
Step 1: Define function template maximum(T a, T b) that returns the larger value.
Step 2: Define function template swapValues(T &a, T &b) that swaps two values.
Step 3: Define function template showPair(T first, U second) with two type
        parameters.
Step 4: Define class template Calculator<T> with add(), subtract(), multiply()
        and divide() member functions.
Step 5: Define class template Stack<T> with push(), pop() and isEmpty()
        member functions.
Step 6: In main(), call the function templates with int, double, char and
        string arguments.
Step 7: Create objects Calculator<int>, Calculator<double>, Stack<int> and
        Stack<string>, call their member functions and display the results.
STOP
```

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++ (C++11 standard)
IDE / Platform	Terminal / text editor on Linux
Compiler	g++ (GCC)
Libraries / Packages	<iostream> (input/output), <string> (string type)

09. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**
Branch Used	main
Commit ID / Latest Commit	7f8fc4031843bc9245fef14b740b25841f3d265b
Date of Last Commit	24/10/26

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1 maximum() int	10, 20	maximum(10, 20) = 20	maximum(10, 20) = 20	Pass
TC2 maximum() double	3.5, 2.1	maximum(3.5, 2.1) = 3.5	maximum(3.5, 2.1) = 3.5	Pass
TC3 maximum() char	'a', 'z'	maximum('a', 'z') = z	maximum('a', 'z') = z	Pass
TC4 maximum() string	apple, banana	maximum("apple", "banana") = banana	maximum("apple", "banana") = banana	Pass
TC5 swapValues() int	x = 5, y = 10	After swap: x = 10, y = 5	After swap: x = 10, y = 5	Pass
TC6 swapValues() string	Hello, World	After swap: s1 = World, s2 = Hello	After swap: s1 = World, s2 = Hello	Pass
TC7 showPair() two types	1, "Apple"	Pair: (1, Apple)	Pair: (1, Apple)	Pass
TC8 Calculator<int>	20, 4	add = 24, subtract = 16, multiply = 80, divide = 5	add = 24, subtract = 16, multiply = 80, divide = 5	Pass
TC9 Calculator<double>	7.5, 2.5	add = 10, subtract = 5, multiply = 18.75, divide = 3	add = 10, subtract = 5, multiply = 18.75, divide = 3	Pass
TC10 Stack<int>	push 10, 20, 30; pop twice	30, then 20	30, then 20	Pass
TC11 Stack<string>	push C++, Templates; pop twice	Templates, then C++	Templates, then C++	Pass
TC12 Stack underflow	pop from empty stack	Stack underflow	Stack underflow	Pass

11. Results / Observations

- A single maximum() function template worked correctly for int, double, char and string arguments; the compiler generated a separate version for each type.
- The swapValues() template swapped both integers and strings using the same code.
- Calculator<int> performed integer division (20 / 4 = 5), whereas Calculator<double> handled decimal values (7.5 * 2.5 = 18.75), showing that the same class works for different types.
- The Stack class template stored int and string values independently, and the underflow condition was reported correctly when popping from an empty stack.

Screenshots / Graphs / Output:

Practical_assignment_10.cpp > main()

```cpp
#include <iostream>
#include <string>
using namespace std;

// Function template: maximum()
template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

// Function template: swapValues()
template <typename T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

// Function template with two types
template <typename T, typename U>
void showPair(T first, U second) {
    cout << "Pair: (" << first << ", " << second << ")" << endl;
}

// Class template: Calculator
template <class T>
class Calculator {
    T a, b;

public:
    Calculator(T x, T y) : a(x), b(y) {}

    T add() const {
        return a + b;
    }

    T subtract() const {
        return a - b;
    }

    T multiply() const {
        return a * b;
    }

    T divide() const {
        return a / b;
    }
};
```

```cpp
};

// Class template: Stack
template <class T>
class Stack {
    T arr[10];
    int top;

public:
    Stack() : top(-1) {}

    void push(T value) {
        if (top == 9) {
            cout << "Stack overflow" << endl;
            return;
        }

        arr[++top] = value;
    }

    T pop() {
        if (top == -1) {
            cout << "Stack underflow" << endl;
            return T();
        }

        return arr[top--];
    }

    bool isEmpty() const {
        return top == -1;
    }
};

int main() {
    cout << "===== 1. Function template: maximum() =====" << endl;

    cout << "maximum(10, 20) = " << maximum(10, 20) << endl;
    cout << "maximum(3.5, 2.1) = " << maximum(3.5, 2.1) << endl;
    cout << "maximum('a', 'z') = " << maximum('a', 'z') << endl;
    cout << "maximum(\"apple\", \"banana\") = "
         << maximum(string("apple"), string("banana")) << endl;

    cout << endl << "===== 2. Function template: swapValues() =====" << endl;

```

```
84
85      cout << "maximum(10, 20) = " << maximum(10, 20) << endl;
86      cout << "maximum(3.5, 2.1) = " << maximum(3.5, 2.1) << endl;
87      cout << "maximum('a', 'z') = " << maximum('a', 'z') << endl;
88      cout << "maximum(\"apple\", \"banana\") = "
89           << maximum(string("apple"), string("banana")) << endl;
90
91      cout << endl << "===== 2. Function template: swapValues() =====" << endl;
92
93      int x = 5, y = 10;
94
95      cout << "Before swap: x = " << x << ", y = " << y << endl;
96      swapValues(x, y);
97      cout << "After swap: x = " << x << ", y = " << y << endl;
98
99      string s1 = "Hello", s2 = "World";
100
101     cout << "Before swap: s1 = " << s1 << ", s2 = " << s2 << endl;
102     swapValues(s1, s2);
103     cout << "After swap: s1 = " << s1 << ", s2 = " << s2 << endl;
104
105     cout << endl << "===== 3. Function template with two types =====" << endl;
106
107     showPair(1, "Apple");
108     showPair(2.5, 'A');
109
110     cout << endl << "===== 4. Class template: Calculator<T> =====" << endl;
111
112     Calculator<int> ci(20, 4);
113
114     cout << "Calculator<int>(20, 4):" << endl;
115     cout << "add = " << ci.add()
116          << ", subtract = " << ci.subtract()
117          << ", multiply = " << ci.multiply()
118          << ", divide = " << ci.divide() << endl;
119
120     Calculator<double> cd(7.5, 2.5);
121
122     cout << "Calculator<double>(7.5, 2.5):" << endl;
123     cout << "add = " << cd.add()
124          << ", subtract = " << cd.subtract()
125          << ", multiply = " << cd.multiply()
126          << ", divide = " << cd.divide() << endl;
127
128     cout << endl << "===== 5. Class template: Stack<T> =====" << endl;
129
130     Stack<int> intStack;
```

```
127
128     cout << endl << "===== 5. Class template: Stack<T> =====" << endl;
129
130     Stack<int> intStack;
131
132     intStack.push(10);
133     intStack.push(20);
134     intStack.push(30);
135
136     cout << "Stack<int> pop: " << intStack.pop() << endl;
137     cout << "Stack<int> pop: " << intStack.pop() << endl;
138
139     Stack<string> strStack;
140
141     strStack.push("C++");
142     strStack.push("Templates");
143
144     cout << "Stack<string> pop: " << strStack.pop() << endl;
145     cout << "Stack<string> pop: " << strStack.pop() << endl;
146
147     strStack.pop();  // Empty stack
148
149     return 0;
150 }
```

- Output:-

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== 1. Function template: maximum() =====
maximum(10, 20) = 20
maximum(3.5, 2.1) = 3.5
maximum('a', 'z') = z
maximum("apple", "banana") = banana

===== 2. Function template: swapValues() =====
Before swap: x = 5, y = 10
After swap: x = 10, y = 5
Before swap: s1 = Hello, s2 = World
After swap: s1 = World, s2 = Hello

===== 3. Function template with two types =====
Pair: (1, Apple)
Pair: (2.5, A)

===== 4. Class template: Calculator<T> =====
Calculator<int>(20, 4):
add = 24, subtract = 16, multiply = 80, divide = 5
Calculator<double>(7.5, 2.5):
add = 10, subtract = 5, multiply = 18.75, divide = 3

===== 5. Class template: Stack<T> =====
Stack<int> pop: 30
Stack<int> pop: 20
Stack<string> pop: Templates
Stack<string> pop: C++
Stack underflow
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

In this practical, function templates and class templates were implemented in C++. Generic functions (maximum, swapValues, showPair) and generic classes (Calculator and Stack) were written once and used with int, double, char and string types. I learned that templates support generic programming, avoid code duplication and make programs reusable and type safe.

13. Assignment Questions

Q1. What is a template in C++? What are its types?

Answer: A template is a C++ feature that allows a function or a class to work with any data type by using a generic type parameter. The compiler generates the required version of the code for each type used. The two types of templates are **function templates** (generic functions, such as maximum(T a, T b)) and **class templates** (generic classes, such as Stack<T>).

Q2. What are the advantages of using templates?

Answer: Templates provide code reusability, because the same code works for many data types. They avoid code duplication and make maintenance easier. They are type safe, since type checking is done at compile time, and they are resolved at compile time, so there is no runtime overhead.

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Pratik Akrur Pandit	202501110116	